

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

Complete Engineering Reference | Innomatics Research Labs | May 2026

Purpose	Complete system explanation for engineering review
Stack	Python 3.10+, LangChain, LangGraph, ChromaDB, HuggingFace, OpenAI
Pattern	RAG + Graph-based Workflow + HITL Escalation
Date	May 2026

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI design pattern that enhances a Large Language Model by giving it access to an external knowledge base at inference time. Instead of relying solely on information baked into its training weights, the LLM first retrieves the most relevant documents from a vector database and then uses them as context to generate a grounded, accurate answer.

This prevents hallucination — the tendency of LLMs to confidently generate incorrect information. RAG grounds the answer in real, verifiable source documents.

Why is RAG needed for Customer Support?

- Support knowledge bases change frequently — RAG uses live documents without retraining
- Customers ask highly specific questions that generic LLMs cannot answer from memory
- Answers must be traceable to source documents for compliance and auditability
- HITL escalation provides a safety net when AI confidence is low

Use Case Overview:

A customer support bot for a company that maintains product manuals, FAQs, and policy documents as PDFs. Users ask questions in natural language. The bot retrieves relevant policy/FAQ sections, generates a concise answer, and escalates to a live agent when it cannot confidently respond.

2. System Architecture Explanation

The system is split into two pipelines: an offline ingestion pipeline (runs once) and an online query pipeline (runs per user request). Both are orchestrated by LangGraph.

Offline Pipeline — Knowledge Base Ingestion:

PDFs are loaded by PyPDFLoader and split into overlapping 500-character chunks by RecursiveCharacterTextSplitter. Each chunk is encoded into a 384-dimensional vector by the HuggingFace all-MiniLM-L6-v2 model and stored in ChromaDB on disk. This runs once — subsequent runs reuse the persisted ./db directory.

Online Pipeline — Query Processing:

On each user query, the LangGraph workflow is invoked. The process_node calls retrieve_docs(), which performs semantic similarity search against ChromaDB. The top-3 results are evaluated by score. If the best score is acceptable (≤ 1.5), the most relevant text line is passed to generate_answer() along with the original query. GPT-4o-mini generates a factual response. If retrieval fails or score is too high, escalate=True is set and the query is passed to human_fallback() for manual handling.

Component Interactions:

Interaction	Description
main.py → ingest.py	main checks if ./db exists; if not, calls ingest_pdfs() once
main.py → graph.py	build_graph() returns compiled LangGraph app; invoked per query
graph.py → retrieval.py	process_node calls state['retriever'](query) = retrieve_docs()
graph.py → generator.py	process_node calls state['generator'](query, context) — currently context is returned directly
graph.py → hitl.py	output_node calls state['hitl'](query) = human_fallback() when escalate=True
retrieval.py → ChromaDB	Shared global vectorstore object; initialized at module import

3. Design Decisions

Chunk Size: 500 chars, Overlap: 100:

500 characters balances context completeness with retrieval precision. Too large → noisy retrieval; too small → loses sentence context. 100-char overlap ensures sentences split across chunk boundaries are still captured.

Embedding Strategy: all-MiniLM-L6-v2:

Chosen for zero cost, local execution, and strong semantic similarity performance on English text. 384 dimensions are compact yet expressive. No API call needed for encoding.

Retrieval Approach: Cosine Distance with Score Threshold:

Cosine similarity is standard for dense vector retrieval. Score threshold of 1.5 was chosen empirically — scores above this indicate the query and document are semantically unrelated. This gates the LLM from generating answers with irrelevant context.

Prompt Design Logic:

The prompt separates Context and Question clearly. Temperature=0.3 ensures the model stays factual and does not creatively extrapolate beyond the retrieved content. System role is set as 'customer support assistant' to tune the response style.

4. Workflow Explanation

LangGraph Usage:

LangGraph is used instead of a simple Python function chain because it provides explicit state management, clear node separation, and makes the HITL escalation path easy to extend in future (e.g., adding more routing nodes, retry logic, or audit logging).

Node Responsibilities:

`process_node` is responsible for all retrieval logic: calling the retriever, evaluating context, and setting the escalation flag. It does not produce the final output — it only populates state. `output_node` is the decision point: it either returns the AI answer or delegates to the human fallback.

State Transitions:

```
START
↓
process_node(state)
→ retrieve_docs(query)
→ if context found: state['answer'] = context, state['escalate'] = False
→ if not found: state['escalate'] = True
↓
output_node(state)
→ if escalate == True: state['answer'] = human_fallback(query)
→ if escalate == False: return state as-is
↓
END → print state['answer']
```

5. Conditional Logic

Intent Detection:

Intent is implicitly detected through semantic similarity. If the query's embedding is close to a chunk in the knowledge base, the system assumes the query is answerable. No explicit NLP intent classification is performed — the cosine score acts as the confidence proxy.

Routing Decisions:

The single routing decision is binary: escalate or not. This is determined by whether `retrieve_docs()` returns a non-None, non-empty string. All edge cases (empty db, bad PDF, LLM timeout) collapse to None return → escalation, making the system fail-safe.

6. HITL Implementation

Role of Human Intervention:

HITL serves as the safety net of the system. When the AI cannot find relevant context or fails to generate a response, a human support agent is notified. The agent sees the exact query and types a response that is delivered to the user in the same format as an AI answer.

Benefits:

- Prevents incorrect AI answers from reaching customers
- Maintains user trust by always providing an answer
- Human responses can be logged to improve the knowledge base over time
- Zero latency penalty in non-escalation path

Limitations:

- Requires a human to be available at the terminal (not production-ready as-is)
- No ticketing system or async escalation — synchronous blocking call
- Human response is not stored back into the knowledge base automatically

7. Challenges and Trade-offs

Retrieval Accuracy vs Speed:

Higher k (more chunks retrieved) improves recall but increases LLM prompt size and cost. k=3 is the balance point — captures most relevant context without bloating the prompt.

Chunk Size vs Context Quality:

Larger chunks give the LLM more context but reduce retrieval precision (noisy matches). Smaller chunks are precise but may cut off mid-sentence. 500 chars is the pragmatic middle ground.

Cost vs Performance:

GPT-4o-mini is used over GPT-4 for cost efficiency. For production use with higher accuracy requirements, GPT-4 or Claude could replace it with a configuration change.

Local Embedding vs API Embedding:

all-MiniLM-L6-v2 runs locally — zero cost but limited to English and generic domains. OpenAI text-embedding-3-small would offer better multilingual support at API cost.

ChromaDB vs Cloud Vector Store:

ChromaDB is perfect for development and small scale. For production with millions of chunks, Pinecone, Weaviate, or Qdrant would offer distributed search and better concurrency.

8. Testing Strategy

Testing Approach:

- Unit test each module independently (ingest, retrieve, generate, hitl, graph)
- Test retrieval with known queries that should/should not match the knowledge base
- Test HITL path by passing queries with no relevant context
- Test graph invocation with mock retriever and generator functions
- End-to-end test with real PDF and sample queries

Sample Test Queries:

```
# Should retrieve and answer:
'What is the refund policy?'
'How do I reset my password?'
'What are the shipping charges?'

# Should escalate to HITL (not in KB):
'What is the CEO's email?'
'Can I get a discount code?'
'What is your company's annual revenue?'
```

9. Future Enhancements

Multi-document Support: Already architected — ingest_pdfs() accepts a list. Add a UI to upload new PDFs dynamically.

Feedback Loop: Log escalated queries. Use them to identify knowledge gaps and update the PDF knowledge base.

Memory Integration: Add conversation history to the state dict. Pass previous Q&A pairs as context for multi-turn support.

Deployment: Wrap main.py in a FastAPI server. Containerize with Docker. Deploy on AWS/GCP with ChromaDB replaced by Pinecone.

Async HITL: Replace terminal input() with a Slack/email notification system for async human response integration.